Coefficient relocation: Lean invariant and native diagnostic

Lean proves the copy invariant for the observed index snapshot. Three operational updates copy old basis indices[0,2,0] into matrix indices[2,3,4]. Matrix lookup preserves each coefficient for every basis content. The rewritten basis lookup fails the live-slot contract at3; rewritten slot2 can be in range and still denote a different coefficient. The universal wrong-table invariant is directly negated. [pcc:unverified]

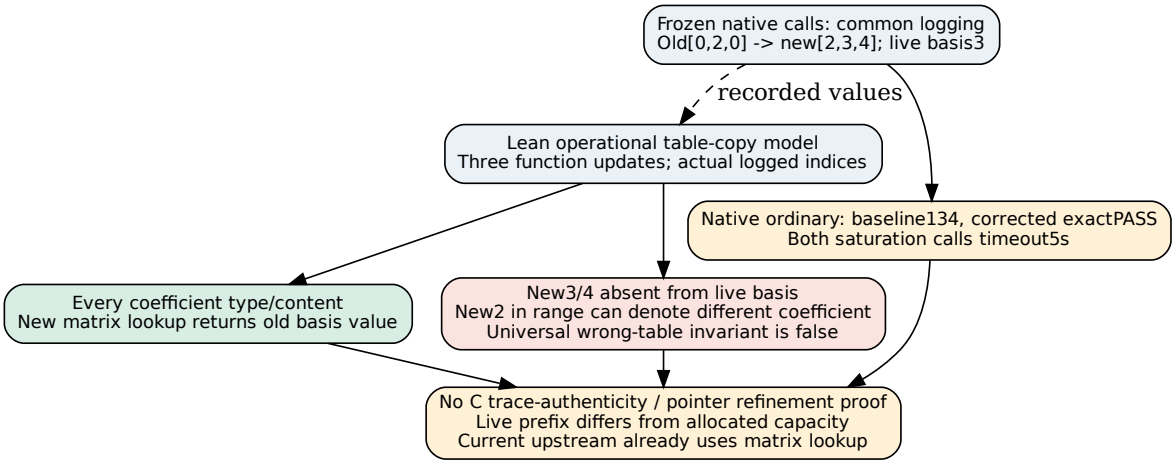

Figure 1: Operational proof, actual logged values and separate native observations. [pcc:unverified]

Actual calls, exact proof obligations and scope

Common-logged source variant	Ordinary exactLA2	Saturation exactLA2
Original basis lookup	abort134; lookup3 with live basis3	timeout124 at5s
Corrected matrix lookup	exit0; exact known basis passes	timeout124 at5s

The variants differ only in one known-pivot coefficient lookup after identical logging and address-only build flags. In both ordinary logs, source2 is copied to matrix3 while the live basis count is3. The old variant then reads basis3. The corrected variant reads matrix3 and its complete known-answer basis passes: input reductions, S-pairs, staircase3 and three Boolean-candidate zeros. Both saturation failures remain unvalidated. This is a fixed-fixture result. [pcc:unverified]

The Lean table is defined by actual sequential function updates, before its copy invariant is proved. Finite row cases prove the invariant for all coefficient contents. The live-basis wrapper returns none at3/4; the matrix returns source entries2/0. A natural-valued basis supplies the in-range counterexample at2. No conclusion is assumed as a premise. [pcc:unverified]

Declaration	Obligation
copied_lookup_correct	Operational copy followed by matrix lookup
observed_live_index_failures	Both nonlive basis indices and correct copied values
in_range_lookup_can_change_coefficients	Wrong coefficient can occur despite an in-range new index
universal_basis_reindex_invariant_false	Direct negation of universal wrong-table equality
reindex_audit_certificate	Copy, mismatch, negation and representability arithmetic

The live prefix differs from allocated C capacity; no physical array bounds assertion is inferred. The native trace's authenticity and the C parser, initialization, pointer semantics and whole F4Run refinement are separate obligations. The earlier signed-shift issue remains unrepaired. The integer representability lemmas do not claim to model all C signed-shift semantics. [pcc:unverified]

Pinned0.6.5's16bit reducer and current upstream32bit reducer already use the matrix table. This is a backport/source audit, with no upstream novelty claim. Primary code URLs, exact current-source hash and read scopes are retained. Native protocolversion3/hash99beb128 was committed at bf2ce5da before all four calls. Flow: 4 planned, 4 attempted, 2 completed, 2 timeouts and 0 excluded. All raw files and prior failures remain retained. Native packet4867bb is not_covered and unsigned. [pcc:unverified]

Lean4.35.0 / Mathlib56a00585: full gate14declarations, zero failures and zero trusted-native exceptions. Only propext, Classical.choice and Quot.sound; no holes, custom axioms, native_decide, waivers or increased elaboration limits. Exact source plan, actual local replay and independent signed standing are recorded separately in sources.json/verification.json. No paid compute, timing, statistical or independent-host claim. KB unavailable, wrapper127; bounded source/GCC/Mathlib fallback. [pcc:unverified]